GOA INSTITUTE OF MANAGEMENT
PGDM – Big Data Analytics (BDA) · Term VI · AY 2026–27

MLOps

Operationalizing Machine Learning Systems

Student Handbook

A Complete Reference

Dr. Alok Tiwari
alok@gim.ac.in

Covering all 24 sessions across 12 modules.
Aligned to all learning outcomes CLO1–CLO6

Tools covered: Git · DVC · MLflow · Docker · FastAPI · GitHub Actions

Preface

Welcome to the MLOps Student Handbook.

This handbook is your primary academic companion for the course *MLOps – Operationalizing Machine Learning Systems* offered in Term VI of the PGDM–BDA programme at Goa Institute of Management. It has been designed to serve three simultaneous purposes: as a structured reading companion for all 24 sessions, as a conceptual reference for lab work, and as a thorough revision resource before the Exams.

How This Handbook Is Organised

The handbook is organised into twelve modules, each corresponding to a pair of course sessions. Every module begins with learning objectives and a session overview, then develops the conceptual content in full, and concludes with an Examination Focus box that explicitly highlights the ideas most likely to appear in assessments.

- **Modules 1–2** (Sessions 1–4): Foundations of MLOps and Reproducibility
- **Modules 3–4** (Sessions 5–8): Version Control, Experiment Tracking, and Model Management
- **Modules 5–6** (Sessions 9–12): Feature Pipelines and Model Packaging
- **Modules 7–8** (Sessions 13–16): Deployment and CI/CD
- **Modules 9–10** (Sessions 17–21): Monitoring, Drift, and End-to-End Pipelines
- **Modules 11–12** (Sessions 22–24): Cloud MLOps, Governance, and Ethics

A Note on Diagrams

Throughout this handbook, conceptual diagrams are presented in structured text form to support pen-and-paper drawing in examinations. Students are strongly encouraged to practise reproducing these diagrams freehand, as the ability to sketch clean, labelled architecture diagrams is explicitly assessed in the Exams.

Tools and Platforms

This course uses exclusively free and open-source tools. Where code examples appear in the text, they are for conceptual illustration only. No coding is required for the Exams; however, understanding *what* tools do and *why* they are used is essential.

Dr. Alok Tiwari
February 2026

Contents

Preface **3**

1 Foundations of MLOps **9**
1.1 The Machine Learning Lifecycle . . . 9
1.2 ML Model vs ML System . . . 10
1.2.1 Why ML Systems Fail in Practice . . . 11
1.3 What is MLOps? . . . 11
1.3.1 The Three Axes of MLOps . . . 11
1.4 MLOps Maturity Levels . . . 12
1.4.1 Diagnosing a Low-Maturity Organisation . . . 13
1.5 Notebook-Driven vs Config-Driven ML Workflows . . . 13
1.6 The Business Case for MLOps . . . 14

2 Reproducibility and Environment Management **17**
2.1 Defining Reproducibility in ML . . . 17
2.2 The Four Sources of Reproducibility Failure . . . 17
2.2.1 Source 1: Code Variability . . . 18
2.2.2 Source 2: Data Variability . . . 18
2.2.3 Source 3: Environment Variability . . . 18
2.2.4 Source 4: Configuration / Hyperparameter Variability . . . 18
2.3 Python Environment Management . . . 19
2.3.1 Key Environment Management Tools . . . 19
2.4 Structuring an ML Project . . . 19
2.4.1 The Role of Configuration Files . . . 20

3 Version Control for Code, Data, and Models **21**
3.1 Git and GitHub for ML Workflows . . . 21
3.1.1 Core Git Concepts . . . 21
3.1.2 Git Branching Best Practices for ML . . . 21
3.1.3 What Should and Should Not Be Committed . . . 22
3.2 Dataset Versioning: Why Files Alone Are Not Enough . . . 23
3.3 DVC: Data Version Control . . . 23
3.3.1 How DVC Works . . . 23
3.3.2 The Linkage Chain: Code, Data, and Experiments . . . 24
3.3.3 Why This Linkage Is Critical . . . 24
3.4 Limitations of Ad-Hoc Model Storage . . . 24

4 Experiment Tracking and Model Management **27**
4.1 Experiment Tracking: The Problem It Solves . . . 27
4.2 MLflow: Core Concepts . . . 27
4.2.1 The Four Core Tracking Entities . . . 27
4.2.2 Experiments and Runs . . . 28
4.2.3 Why Experiment Tracking Matters for Teams . . . 29
4.3 The Model Registry . . . 29
4.3.1 The Model Lifecycle . . . 29
4.3.2 Governance at Stage Transitions . . . 30

5 Feature Pipelines and Data Validation **31**
5.1 Feature Pipelines in ML Systems . . . 31
5.2 Training–Inference Skew . . . 31
5.2.1 How Training–Inference Skew Arises . . . 31
5.2.2 Preventing Training–Inference Skew: Shared Pipeline Design . . . 32
5.3 Schema Validation . . . 33
5.3.1 Schema Mismatch as a Production Failure Mode . . . 33

6 Model Packaging and Containerisation **35**
6.1 Model Serialisation . . . 35
6.1.1 Serialisation Pitfalls . . . 35
6.2 Docker for ML: Concepts . . . 36
6.2.1 Key Docker Concepts . . . 36
6.2.2 Dockerfile Structure for ML Inference Services . . . 37
6.2.3 How Containerisation Solves Environment Inconsistency . . . 37

7 Model Serving and Deployment **39**
7.1 Deployment Patterns: Batch vs Real-Time Inference . . . 39
7.1.1 Batch Inference . . . 39
7.1.2 Real-Time Inference (Online Inference) . . . 39
7.1.3 Comprehensive Comparison . . . 40
7.1.4 Choosing Between Batch and Real-Time . . . 40
7.2 REST APIs for ML: The FastAPI Pattern . . . 40
7.2.1 Architecture of a FastAPI ML Inference Service . . . 41
7.2.2 Production Failure Points in the Inference Pipeline . . . 41

8 CI/CD for Machine Learning **43**
8.1 CI/CD: From Software Engineering to Machine Learning . . . 43
8.2 How ML CI/CD Differs from Software CI/CD . . . 43
8.3 The ML CI/CD Pipeline . . . 44
8.3.1 The Evaluation Gate: The Core Governance Mechanism . . . 45
8.3.2 GitHub Actions for ML Pipelines . . . 46

9 Monitoring, Drift, and Retraining **47**
9.1 Why Monitoring Is Non-Negotiable . . . 47
9.2 Data Drift and Concept Drift . . . 47
9.2.1 Data Drift (Covariate Shift) . . . 48
9.2.2 Concept Drift (Label Drift) . . . 48
9.3 Designing a Monitoring Framework . . . 49
9.3.1 What to Monitor: Three Metric Categories . . . 49
9.3.2 Logging and Alerting Strategy . . . 50
9.3.3 Retraining Trigger Policy . . . 50
9.4 The Risk Argument for Monitoring in Safety-Adjacent Systems . . . 51

10 End-to-End MLOps Pipeline **53**
10.1 Integrating the Full MLOps Stack . . . 53
10.2 The Seven Layers of an MLOps Architecture . . . 53
10.3 How the Layers Connect . . . 54
10.4 Governance Integration Points . . . 55

11 Cloud MLOps Concepts **57**
11.1 Cloud MLOps: The Conceptual Shift . . . 57
11.2 Cloud-to-Open-Source Mapping . . . 57

11.3 Scaling Considerations 57

12 Governance, Ethics, and Industry Readiness **59**
12.1 Responsible AI in Production ML 59
12.1.1 Fairness 59
12.1.2 Privacy 61
12.1.3 Accountability 61
12.1.4 Transparency 61
12.2 Model Auditability vs Model Explainability 62
12.3 Designing a Minimum Audit Trail 62
12.4 MLOps Organisational Roles 63
12.4.1 RACI Framework for ML Model Deployment 63

Appendix: Key Concepts Glossary **65**

Chapter 1

Foundations of MLOps

Sessions 1–2 · CLO1

Module 1 at a Glance

Sessions: 1 & 2 **Mode:** Lecture **CLO:** CLO1

Topics: ML systems vs ML models; limitations of notebook-driven workflows; the MLOps lifecycle; the tooling ecosystem overview; business motivation for operational ML.

By the end of this module, you should be able to: explain the complete ML lifecycle, distinguish between an ML model and an ML system, identify MLOps maturity gaps in real-world scenarios, and articulate the business case for investing in MLOps infrastructure.

1.1 The Machine Learning Lifecycle

The term **machine learning lifecycle** refers to the complete set of stages a machine learning initiative passes through, from the first identification of a business problem to the retirement of a deployed model. Understanding this lifecycle is foundational to MLOps because MLOps is, at its core, the discipline of managing this lifecycle in a rigorous, repeatable, and scalable manner.

The lifecycle can be understood as having six broad phases:

1. **Problem Framing:** Translating a business objective into an ML problem definition. This includes determining whether ML is the right solution, defining success metrics, and identifying data requirements.

2. **Data Acquisition and Preparation:** Sourcing, labelling, cleaning, and transforming data into a form suitable for training. This phase is often the most time-consuming in practice.

3. **Experimentation and Model Development:** Training candidate models, tuning hyperparameters, comparing approaches, and selecting the best-performing model according to defined metrics.

4. **Model Evaluation and Validation:** Rigorously assessing model performance on held-out data, checking for fairness issues, and validating against business acceptance criteria.

5. **Deployment:** Packaging the trained model and making it available for inference, whether as a batch job, a REST API, or an embedded component.

6. **Monitoring and Retraining:** Continuously observing model behaviour in production, detecting degradation, and triggering retraining when necessary.

These six phases are not strictly linear. In practice, they form a **feedback loop**: insights from monitoring in production feed back into problem re-framing, data collection, and model retraining. This cyclical nature is why MLOps is often depicted as a continuous loop rather than a waterfall pipeline.

Conceptual Diagram: The ML Lifecycle Loop

```
+------------------+
|  Problem Framing |<-----------+
+--------+---------+            |
         |                      |
         v                      |
+------------------+            |
| Data Acquisition |            |
+--------+---------+            |
         |                      |
         v                      |
+------------------+   FEEDBACK |
| Experimentation  |    LOOP    |
+--------+---------+            |
         |                      |
         v                      |
+------------------+            |
|  Evaluation &    |            |
|   Validation     |            |
+--------+---------+            |
         |                      |
         v                      |
+------------------+            |
|   Deployment     |            |
+--------+---------+            |
         |                      |
         v                      |
+------------------+            |
| Monitoring &     +------------+
|   Retraining     |
+------------------+
```

1.2 ML Model vs ML System

One of the most important conceptual distinctions in this course is the difference between an **ML model** and an **ML system**. This distinction is often overlooked by students with a purely technical background, but it is central to understanding why MLOps is necessary.

> **ML Model:** A mathematical function, typically parameterised by weights or rules, that maps input features to a predicted output. A model is the artefact produced by a training algorithm. It lives in memory or on disk. By itself, it does nothing.
> **ML System:** The complete sociotechnical infrastructure that surrounds a model to make it useful in a real-world context. An ML system includes the data pipelines that feed the model, the feature engineering logic, the serving infrastructure, the monitoring components, the retraining workflows, the access controls, the logging systems, and the human processes that govern its use.

The critical insight, which is often attributed to practitioners at Google, is that *the model is a tiny part of the ML system*. Researchers at Google estimated that in a mature ML

deployment, only about 5% of code is the actual modelling logic. The remaining 95% is the surrounding system: configuration, data collection, feature extraction, process management, serving infrastructure, and monitoring.

▷ **Classroom Note:** Think of an ML model like the engine of a car. The engine is essential, but you cannot drive anywhere without the chassis, wheels, fuel system, controls, and road infrastructure. MLOps is the discipline of building and maintaining all of that supporting infrastructure for ML.

1.2.1 Why ML Systems Fail in Practice

Understanding failure modes is central to the MLOps mindset. ML initiatives fail for a range of reasons that go far beyond model accuracy:

- **Reproducibility failures:** The model cannot be retrained to produce the same results, so fixes are impossible.
- **Deployment failures:** The model works on a data scientist's laptop but breaks in the production environment due to library conflicts or missing dependencies.
- **Monitoring gaps:** The model is deployed but no one observes its predictions, so performance degradation goes unnoticed for months.
- **Data pipeline drift:** The production data fed to the model changes in distribution over time, but the model was trained on historical data that no longer reflects reality.
- **Governance failures:** No record exists of which data was used to train a deployed model, making regulatory audits impossible.
- **Organisational silos:** Data scientists build models that data engineers cannot deploy, and operations teams cannot monitor.

1.3 What is MLOps?

Machine Learning Operations (MLOps) is a set of practices, tools, and cultural norms that aim to unify the development (Dev) and operation (Ops) of machine learning systems. Borrowing from the DevOps movement in software engineering, MLOps adapts continuous integration, continuous delivery, and monitoring principles to the specific challenges of ML workflows.

A precise definition: MLOps is the discipline of applying software engineering best practices — including version control, automated testing, continuous integration, continuous delivery, and monitoring — to the complete machine learning lifecycle, with the goal of making ML systems reliable, reproducible, scalable, and governable in production.

1.3.1 The Three Axes of MLOps

MLOps operates along three axes simultaneously:

1. **Code:** Is the modelling code version-controlled, tested, and deployable in an automated fashion?
2. **Data:** Are the datasets version-controlled? Is data quality validated before training? Is there a record of which data produced which model?

3. **Models:** Are trained models tracked, versioned, and governed? Is there a clear lifecycle from experimental model to production model to retired model?

Weak MLOps typically means one of these axes is managed informally or not at all. Mature MLOps means all three axes are automated, version-controlled, and governed.

1.4 MLOps Maturity Levels

The concept of a **MLOps maturity model** provides organisations with a framework to understand their current state and plan improvement. Different frameworks use different scales, but a three-level model is widely understood and sufficient for this course.

MLOps Maturity Ladder: Three Levels

Level	Characteristics	Typical Symptoms
Level 0: Manual	Notebooks, manual steps, no versioning, no monitoring	Long deployment cycles, unreproducible results, production incidents with no rollback capability
Level 1: ML Pipeline Automation	Automated training pipelines, experiment tracking, model registry, basic monitoring	Reproducible training; still manual trigger for retraining; some governance gaps
Level 2: CI/CD for ML	Fully automated pipelines triggered by data or code changes, automated testing, continuous monitoring, feedback loops	Near-continuous deployment; model quality gates enforce standards; full auditability

Diagram: MLOps Maturity Ladder

```
Automation
Level
   ^
   |
L2 |  +---------------------------------------------+
   |  |  CI/CD for ML                               |
   |  |  Automated pipelines, quality gates,        |
   |  |  continuous monitoring, full auditability   |
   |  +---------------------------------------------+
   |
L1 |  +---------------------------------------------+
   |  |  ML Pipeline Automation                     |
   |  |  Experiment tracking, model registry,       |
   |  |  automated training, basic monitoring       |
   |  +---------------------------------------------+
   |
L0 |  +---------------------------------------------+
   |  |  Manual (Notebook-Driven)                   |
```

```
|  |  No versioning, no tracking, no monitoring, |
|  |  manual deployment                          |
|  +---------------------------------------------+
+-----------------------------------------------------> Maturity / Time
```

1.4.1 Diagnosing a Low-Maturity Organisation

An organisation at Level 0 typically exhibits the following observable symptoms:

- **Notebook proliferation:** Models live in Jupyter notebooks with names like `model_v3_final_FINAL2.ipynb`.
- **No reproducibility:** Re-running the same notebook on a different machine or at a different time produces different results.
- **Manual deployment:** A data scientist emails a pickle file to the engineering team, who manually copies it to a server.
- **No monitoring:** Once deployed, the model is essentially forgotten until a user complaint surfaces.
- **No record of training data:** No one can say which exact dataset snapshot was used to train the currently-deployed model.
- **Long time-to-deploy:** Moving a model from notebook to production takes weeks of manual effort.

1.5 Notebook-Driven vs Config-Driven ML Workflows

A recurring theme in MLOps is the *limitation of notebooks as a production tool*. Jupyter notebooks are excellent for exploratory data analysis and rapid prototyping, but they are poorly suited to the demands of production ML systems.

Comparison: Notebook-Driven vs Config-Driven Workflow

Dimension	Notebook-Driven	Config-Driven / Modular
Reproducibility	Low: hidden state, execution order sensitivity, environment not captured	High: config files pin every parameter; environment captured via `requirements.txt` or Docker
Collaboration	Poor: merge conflicts, no easy code review, cell outputs committed to git	Good: modular Python scripts enable standard code review and branching workflows
CI/CD Readiness	None: notebooks cannot be easily triggered by CI pipelines	High: scripts are directly callable by GitHub Actions or other CI tools
Auditability	Low: no formal record of runs; output cells may be stale	High: every run logged in experiment tracker; parameters and metrics recorded
Reusability	Low: code tightly coupled to notebook structure	High: modular functions can be imported and reused across pipelines
Testing	Very difficult: no standard way to unit-test notebook cells	Standard: modular code supports unit, integration, and data quality tests

A **config-driven ML project** externalises all hyperparameters, file paths, and environment settings into configuration files (e.g., YAML). This means that two experiments differ only in their configuration files, and both can be tracked, compared, and reproduced exactly.

1.6 The Business Case for MLOps

A common objection from engineering or business leaders is that MLOps infrastructure represents overhead that small or resource-constrained teams cannot afford. This objection, while understandable, conflates short-term cost with long-term risk.

The business case for MLOps rests on four arguments:

1. **Risk reduction:** Unmonitored models silently degrade, leading to incorrect business decisions. In regulated industries (banking, healthcare, insurance), model failures can result in regulatory fines, legal liability, and reputational damage.

2. **Speed and agility:** Teams with mature MLOps infrastructure can deploy a new model in hours rather than weeks. Faster iteration means faster business value.

3. **Auditability and compliance:** Regulators increasingly require organisations to explain AI decisions. Without experiment tracking and model registries, this is impossible.

4. **Scalability:** Manual ML processes do not scale. An organisation running five models manually cannot run fifty models without MLOps automation.

⋆ **Examination Focus:** Examination questions on Module 1 typically ask you to: (a) distinguish ML model from ML system with examples; (b) place an organisation on the maturity ladder based on described symptoms; (c) construct a business case against the argument that MLOps is unaffordable overhead; (d) compare notebook-driven and config-driven workflows across multiple dimensions. Be prepared to draw the maturity ladder diagram and the lifecycle feedback loop.

Chapter 2

Reproducibility and Environment Management

Sessions 3–4 · CLO2

Module 2 at a Glance

Sessions: 3 & 4 **Mode:** Lab **CLO:** CLO2

Topics: Python environments and dependency management; ML project structuring; configuration-driven experimentation; reproducibility failures in practice.

By the end of this module, you should be able to: define reproducibility in ML, identify the four main sources of reproducibility failure, structure an ML project following professional conventions, and explain how configuration files enforce experiment reproducibility.

2.1 Defining Reproducibility in ML

> **Reproducibility** in machine learning means that given the same code, the same data, the same environment, and the same configuration, any team member running the same experiment at any point in time will obtain the same results (within acceptable numerical tolerance for stochastic operations).

Reproducibility is not merely an academic nicety. It is a production-critical property. Without reproducibility:

- You cannot debug a model that misbehaves in production, because you cannot recreate the exact conditions that produced it.
- You cannot comply with audit requirements that ask you to reproduce a specific model's outputs.
- You cannot onboard new team members, because their environment produces different results.
- You cannot safely roll back to a previous model version, because you cannot guarantee the rollback will behave identically.

2.2 The Four Sources of Reproducibility Failure

Reproducibility failures in ML arise from four distinct and independently addressable sources. Understanding each source is essential for designing systems that are reproducible by construction.

2.2.1 Source 1: Code Variability

If the model training code is not version-controlled, different team members may be running different versions of the same script. Even minor code changes (a bug fix, a refactored feature transformation) can produce substantially different model outputs.

Mitigation: Use Git for all code. Every training run should record the exact Git commit hash that produced it.

2.2.2 Source 2: Data Variability

If the dataset used for training is not versioned or pinned, then retraining "on the same data" may silently use a different data snapshot. This is especially problematic when data is stored in live databases that are continuously updated.

Mitigation: Use data versioning tools (DVC, discussed in Module 3) to snapshot and version datasets. Every training run must record the exact dataset version used.

2.2.3 Source 3: Environment Variability

A Python environment is defined by the specific versions of all installed packages. If package versions are not pinned, then the same code run at different times may use different library versions (e.g., scikit-learn 1.2 vs 1.3), which can produce different results due to algorithm changes, bug fixes, or API differences.

Mitigation: Use a `requirements.txt` file with pinned versions, or a `conda environment.yml`. Even better, use Docker containers to freeze the entire environment (covered in Module 6).

2.2.4 Source 4: Configuration / Hyperparameter Variability

If model hyperparameters are hardcoded in notebooks or scripts rather than externalised into configuration files, then there is no reliable record of exactly which hyperparameters produced a given model. A single careless edit to the script can silently change model behaviour.

Mitigation: Externalise all hyperparameters and configuration into YAML or JSON config files. Commit config files to Git alongside code. Use experiment trackers (covered in Module 4) to log config with every run.

Diagram: Four Sources of Reproducibility Failure

```
+-----------------------------------------------------+
|          REPRODUCIBILITY FAILURE SOURCES            |
+------------------+----+---------------------------+
                   |    |
     +-------------+    +--------------+
     |                                 |
[CODE]     [DATA]     [ENVIRONMENT]    [CONFIG]
   |          |            |              |
No version  No dataset  Unpinned       Hardcoded
control     versioning  package        hyperparams
                        versions       in scripts
   |          |            |              |
+-------------------------------------------+
|         MITIGATION STRATEGIES             |
|                                           |
|  Git + commit hash  |  DVC snapshots     |
```

```
|  requirements.txt   |  YAML config files  |
|  Docker containers  |  Experiment tracker |
+-------------------------------------------+
```

2.3 Python Environment Management

A **Python virtual environment** is an isolated Python installation that contains a specific set of packages, independent of the system-wide Python installation and other virtual environments. This isolation is the first line of defence against environment variability.

2.3.1 Key Environment Management Tools

- `venv`: Python's built-in virtual environment tool. Lightweight and sufficient for most projects.
- `conda`: A more powerful environment manager that handles not just Python packages but also system-level dependencies. Popular in the data science community.
- `pip` with `requirements.txt`: The standard way to record and install Python package dependencies. For reproducibility, versions must be *pinned* (e.g., `scikit-learn==1.3.0`, not `scikit-learn>=1.0`).

The key practice is to **always pin dependency versions** in any project intended for production or collaboration. Unpinned dependencies are a reproducibility time bomb.

2.4 Structuring an ML Project

Professional ML projects follow a **standard directory structure** that separates data, code, configuration, models, and outputs into clearly defined locations. This structure makes projects navigable, testable, and CI/CD-compatible.

Standard ML Project Directory Structure

```
my_ml_project/
|
+-- data/
|   +-- raw/          (original, immutable data; never edited)
|   +-- processed/    (cleaned and feature-engineered data)
|
+-- src/
|   +-- data_prep.py   (data loading and preprocessing logic)
|   +-- features.py    (feature engineering functions)
|   +-- train.py       (model training script)
|   +-- evaluate.py    (model evaluation script)
|   +-- predict.py     (inference / prediction script)
|
+-- models/            (serialised trained models)
+-- notebooks/         (exploratory notebooks; NOT production code)
+-- tests/             (unit and integration tests)
+-- configs/
|   +-- train_config.yaml   (hyperparameters and training settings)
|   +-- data_config.yaml    (data paths and schema settings)
|
+-- requirements.txt   (pinned Python dependencies)
```

```
+-- .gitignore         (prevents large data files being committed)
+-- README.md
```

2.4.1 The Role of Configuration Files

In a config-driven workflow, a YAML configuration file specifies all parameters that control an experiment. A conceptual example:

Example: train_config.yaml

```yaml
model:
  type: RandomForestClassifier
  n_estimators: 200
  max_depth: 8
  random_state: 42

data:
  train_path: data/processed/train_v3.csv
  test_path: data/processed/test_v3.csv
  target_column: default_risk

evaluation:
  primary_metric: f1_score
  threshold: 0.5
```

This approach means that every experiment is fully defined by its config file. To run a different experiment, you create a new config file. Both config files are committed to Git. The experiment tracker (covered in Module 4) logs the config file content with every run, creating a complete and searchable experiment record.

⋆ **Examination Focus:** For examinations, be ready to: (a) define reproducibility and explain why it is production-critical; (b) identify and explain the four sources of reproducibility failure given a scenario; (c) recommend specific mitigation strategies for each source; (d) explain what a config-driven workflow is and how it differs from hardcoding hyperparameters. The ability to sketch the project directory structure is a useful complement to written explanations.

Chapter 3

Version Control for Code, Data, and Models

Sessions 5–6 · CLO2

Module 3 at a Glance

Sessions: 5 & 6 **Mode:** Lab **CLO:** CLO2

Topics: Git and GitHub for ML workflows; dataset versioning concepts; introduction to DVC; linking data, code, and experiments.

By the end of this module, you should be able to: apply Git branching best practices for ML workflows, explain data versioning concepts and the role of DVC, describe how DVC links code, data, and experiment runs, and explain why this linkage is critical for auditability.

3.1 Git and GitHub for ML Workflows

Git is the industry-standard distributed version control system for tracking changes to code and other text files. For ML teams, Git serves as the foundation of all collaboration and reproducibility practices. **GitHub** is a cloud platform that hosts Git repositories and adds collaboration features such as pull requests, code reviews, and GitHub Actions (the CI/CD tool covered in Module 8).

3.1.1 Core Git Concepts

- **Repository (repo):** The complete history of a project, stored as a series of *commits*.
- **Commit:** A snapshot of the entire project at a specific point in time, identified by a unique hash (e.g., `a3f92c1`). Commits are the unit of reproducibility for code.
- **Branch:** A separate line of development. Branches allow parallel work without interfering with the main codebase.
- **Merge / Pull Request (PR):** The process of integrating changes from one branch into another, typically after code review.
- **Tag:** A named pointer to a specific commit, conventionally used to mark model release versions (e.g., `v2.1-production`).

3.1.2 Git Branching Best Practices for ML

A disciplined branching strategy is essential for ML teams because experiments proliferate rapidly. Without clear conventions, repositories become disorganised and it becomes impossible to trace which experimental branch produced which model.

The recommended branching strategy for ML projects:

- **`main` branch:** The production-ready branch. Only code that has been validated and approved is merged here. A tagged commit on `main` corresponds to a model in the registry.
- **`develop` branch:** An integration branch where tested features are assembled before a release.
- **Feature branches** (`feature/add-age-feature`): Short-lived branches for developing a single feature transformation or pipeline improvement.
- **Experiment branches** (`exp/gradient-boosting-v2`): Branches for running a specific modelling experiment. These may or may not be merged back depending on results.
- **Hotfix branches** (`hotfix/fix-null-handling`): Emergency fixes to production behaviour.

Git Branching Diagram: Single Model Release Cycle

```
TIME ------->

main     o---------o-----------o================o  [TAG: v2.0]
          \                   /                /
develop    \    o------o-----o               /
            \  |              \             /
feature/age  o-+               \           /
             |                  \         /
exp/gbm-v1   o----------o--------o------/ (merged: improved F1)
                          \
exp/xgboost   o------------o  (not merged: no improvement)
```

3.1.3 What Should and Should Not Be Committed

A common mistake for ML beginners is committing large data files to Git. Git is optimised for text files, not binary data. Committing large CSV or Parquet files bloats the repository and causes slow clone/pull operations.

Git Commit Policy for ML Projects

Commit to Git	Do NOT Commit to Git
Python source code (.py files)	Raw or processed data files (.csv, .parquet, .hdf5)
Configuration files (.yaml, .json)	Trained model files (.pkl, .joblib, .h5)
Tests	Jupyter notebook outputs / cell results
requirements.txt / environment.yml	Secret keys and credentials
README and documentation	Autogenerated build artefacts
`.dvc` pointer files	Files tracked by DVC

3.2 Dataset Versioning: Why Files Alone Are Not Enough

Code version control solves the code axis of reproducibility. But in ML, the *data* is equally important. Consider the following scenario: you retrain your model after discovering a bug in your feature pipeline. The retrained model performs better. But has performance improved because of the bug fix, or because the underlying training dataset has been updated in the meantime? Without data versioning, you cannot tell.

> **Data Versioning** is the practice of creating immutable, uniquely identified snapshots of datasets at specific points in time, and recording which dataset version was used in each model training run. It enables teams to reproduce any past experiment, trace model behaviour to specific data characteristics, and audit model provenance.

The challenge is scale. A training dataset may be gigabytes or terabytes in size. Storing multiple versions of such data naively (by copying) is both expensive and impractical.

3.3 DVC: Data Version Control

DVC (Data Version Control) is an open-source tool that solves the data versioning problem by separating the *reference* to a dataset from the *content* of that dataset. DVC works alongside Git: while Git tracks the code, DVC tracks the data.

3.3.1 How DVC Works

The core mechanism of DVC is the **.dvc pointer file**:

1. When you add a data file to DVC (`dvc add data/train.csv`), DVC:
   - Computes a content hash (MD5) of the file.
   - Moves the file to a DVC cache (a local storage area).
   - Creates a small `data/train.csv.dvc` text file containing the hash and metadata.
2. The large data file is stored in the *DVC remote* (which can be a cloud bucket, a shared server, etc.).
3. Only the small `.dvc` pointer file is committed to Git.
4. When a team member runs `dvc pull`, DVC fetches the exact file corresponding to the hash in the pointer file.

This means that a Git commit now encodes both the code version (via the commit hash) *and* the data version (via the `.dvc` file contents). You can check out any past Git commit and then run `dvc pull` to restore exactly the data that was used at that point in history.

Diagram: How DVC Links Code, Data, and Experiments

```
+-------------------+          +------------------+
|   GIT REPOSITORY  |          |   DVC REMOTE     |
|                   |          |  (Cloud / NFS)   |
|  src/train.py     |          |                  |
|  configs/v2.yaml  |          |  [hash: a3f9...]  |<--+
|  data/train.csv   |          |  train_v2.csv    |   |
|    .dvc (hash:    +--------->|                  |   |
```

```
|      a3f9...)     |  pointer |  [hash: c72b...]  |<--+--+
|                   |          |  train_v3.csv    |   |  |
+-------------------+          +------------------+   |  |
                                                       |  |
+-----------------------------------------------+     |  |
|   EXPERIMENT TRACKER (MLflow)                 |     |  |
|                                               |     |  |
|  Run #42: git_commit=a3b9c1, data=a3f9...    +-----+  |
|           f1=0.83, params={n_est: 200}        |        |
|                                               |        |
|  Run #43: git_commit=d4e7f2, data=c72b...    +--------+
|           f1=0.81, params={n_est: 100}        |
+-----------------------------------------------+
```

3.3.2 The Linkage Chain: Code, Data, and Experiments

The complete linkage chain that enables full reproducibility and auditability is:

Git Commit Hash $\longleftrightarrow$ DVC Pointer File $\longleftrightarrow$ Dataset Hash $\longleftrightarrow$ Experiment Run Record

Given any experiment run ID from the tracker, you can trace back to the exact code version and the exact dataset used. Given any deployed model, you can answer: *"What code produced this model? What data was it trained on? What were its parameters?"* This full traceability is the foundation of ML auditability.

3.3.3 Why This Linkage Is Critical

- **Debugging:** When a production model fails, you can recreate the exact training conditions to understand what happened.
- **Regulatory audit:** Financial and healthcare regulators increasingly require firms to document exactly what data and processes produced a deployed model. DVC-based linkage makes this possible.
- **Collaboration:** New team members can reproduce any past experiment by checking out the relevant Git commit and running `dvc pull`.
- **A/B testing:** You can compare two model versions knowing they differ only in their hyperparameters, not in their training data.

3.4 Limitations of Ad-Hoc Model Storage

A common informal practice, especially in early-stage teams, is to save trained model files in shared drives (Google Drive, Dropbox, network folders). While this is understandable as an expedient, it introduces severe operational and governance problems:

1. **No versioning:** A shared folder cannot tell you when a model was overwritten, what changed, or how to restore a previous version.
2. **No metadata:** The file contains the model weights but no information about what training data, code, hyperparameters, or evaluation metrics produced it.
3. **No lifecycle management:** There is no concept of "staging" vs "production" vs "archived" status. Any model in the folder could theoretically be deployed.

4. **No access control:** In regulated environments, only authorised personnel should be able to promote a model to production. A shared folder has no such gate.

5. **No reproducibility guarantee:** There is no assurance that the model file corresponds to any specific Git commit or DVC dataset version.

A formal **model registry** (discussed in depth in Module 4) addresses all five of these limitations.

⋆ **Examination Focus:** Key examination topics for Module 3: (a) explain Git branching best practices for ML, including which branch types serve which purpose; (b) explain how DVC works mechanically and draw the pointer file linkage diagram; (c) explain why the linkage between code version, data version, and experiment run record is critical for auditability; (d) articulate five limitations of ad-hoc model storage in a shared drive vs a formal model registry. Be prepared to draw the Git branching diagram for a release cycle.

Chapter 4

Experiment Tracking and Model Management

Sessions 7–8 · CLO2, CLO3

Module 4 at a Glance

Sessions: 7 & 8 **Mode:** Lab **CLO:** CLO2, CLO3

Topics: MLflow for experiment tracking; metrics, parameters, artifacts; model registry; managing model lifecycle stages.

By the end of this module, you should be able to: explain the four core entities tracked by MLflow, design a model registry workflow with lifecycle stages, define the governance checks required at each stage transition, and justify the use of a formal model registry over ad-hoc storage.

4.1 Experiment Tracking: The Problem It Solves

In any serious ML project, a data scientist runs dozens or hundreds of experiments before arriving at a model that meets production requirements. Without a systematic approach to recording these experiments, the result is a chaotic situation:

- Results are scattered across spreadsheets, notebook outputs, and personal notes.
- It is impossible to reproduce the best run from six weeks ago because no one recorded all the settings.
- Comparing two model variants requires manually searching through files.
- When a new team member joins, there is no record of what has already been tried.

Experiment tracking is the practice of automatically recording every training run with its complete context: the exact code version, data version, hyperparameters used, and all output metrics and artefacts. The result is a searchable, comparable, and reproducible history of the entire model development process.

4.2 MLflow: Core Concepts

`MLflow` is an open-source platform for managing the ML lifecycle, with a focus on experiment tracking and model management. It has four core components: Tracking, Projects, Models, and Model Registry. In this course, we focus on Tracking and the Model Registry.

4.2.1 The Four Core Tracking Entities

Every experiment run logged in MLflow is a **Run**. A Run records four types of information:

MLflow: Four Core Tracking Entities

Entity	Definition	Example (Demand Forecasting)
Run	A single execution of a training script, with a unique ID, timestamp, and status	Run #47: gradient boosting model, 2026-02-10 14:32
Parameters	Input configuration values that control the training process; hyperparameters	`n_estimators=200, max_depth=6, learning_rate=0.05`
Metrics	Numeric output values that measure model performance; can be logged over training steps	`RMSE=142.3, MAE=98.7, R2=0.87`
Artifacts	Files produced by or associated with a training run	Trained model file (`model.pkl`), feature importance plot, confusion matrix, data schema

4.2.2 Experiments and Runs

In MLflow, Runs are organised into **Experiments**. An Experiment is a named collection of Runs, typically corresponding to a single modelling objective (e.g., "Demand Forecasting v2 – Tree Models"). By grouping Runs into Experiments, teams can compare many model variants on a common leaderboard.

Diagram: MLflow Experiment Structure

```
EXPERIMENT: "Demand Forecasting v2 -- Tree Models"
|
+-- Run #45  [RandomForest]
|       Parameters: n_estimators=100, max_depth=5
|       Metrics:    RMSE=161.2, R2=0.81
|       Artifacts:  model.pkl, feature_importance.png
|
+-- Run #46  [RandomForest]
|       Parameters: n_estimators=200, max_depth=8
|       Metrics:    RMSE=148.9, R2=0.84
|       Artifacts:  model.pkl, feature_importance.png
|
+-- Run #47  [GradientBoosting]   <-- BEST
|       Parameters: n_estimators=200, max_depth=6, lr=0.05
|       Metrics:    RMSE=142.3, R2=0.87
|       Artifacts:  model.pkl, feature_importance.png
|
+-- Run #48  [XGBoost]
        Parameters: n_estimators=300, max_depth=7, lr=0.1
        Metrics:    RMSE=145.8, R2=0.86
        Artifacts:  model.pkl, feature_importance.png
```

4.2.3 Why Experiment Tracking Matters for Teams

Experiment tracking transforms model development from an individual activity into a *collaborative and auditable process*. Key benefits:

- **Comparison:** The MLflow UI presents all runs as rows in a table, with metrics as columns, making it trivial to identify the best-performing run.
- **Reproducibility:** Any run can be rerun by extracting its logged parameters and applying them to the same code commit and DVC data version.
- **Institutional memory:** The experiment history persists beyond individual team members. When someone leaves, their experimental findings remain accessible.
- **Rollback:** If the "latest" model underperforms in production, the best run from the tracker can be quickly retrieved and redeployed.

4.3 The Model Registry

The **Model Registry** is a centralised catalogue of trained ML models, with versioning, lifecycle management, and governance controls. It is the mechanism by which an experimental model transitions from the data scientist's workbench to production deployment.

4.3.1 The Model Lifecycle

Models progress through a series of lifecycle stages, each representing an increasing level of validation and trust. MLflow's Model Registry defines the following stages:

Model Lifecycle Stages in MLflow

Stage	Meaning	Governance Check Before Promotion
None (Candidate)	Model has been logged from an experiment run but not yet reviewed	N/A (automatic on log)
Staging	Model is under review; being tested in a staging environment	Data Scientist confirms metrics exceed threshold; peer review of training code
Production	Model is actively serving predictions in the live system	Model Risk Officer or senior ML Engineer approves; performance on validation set confirmed; fairness check passed
Archived	Model has been retired from production and superseded	New production model has been tested and confirmed stable; rollback plan documented

Diagram: Model Lifecycle State Transitions

```
              +-------------------+
              |  Candidate / None |
              |  (Logged from Run)|
              +--------+----------+
                       |
                       |  Data Scientist review: metrics OK?
                       v
              +-------------------+
              |     STAGING       |  <-- Tested in non-production
              |                   |      environment; peer review
              +--------+----------+
                       |
                       |  Model Risk Officer / Senior ML Eng approves
                       |  Fairness check, perf validation passed
                       v
              +-------------------+
              |    PRODUCTION     |  <-- Actively serving predictions
              |                   |
              +--------+----------+
                       |
                       |  New production model stable and confirmed
                       v
              +-------------------+
              |     ARCHIVED      |  <-- Retired; preserved for audit
              +-------------------+

NOTE: At any point, a Production model can be rolled back
      to a previous version by promoting an Archived version.
```

4.3.2 Governance at Stage Transitions

The value of the model registry lies in the *governance gates* at each stage transition. These gates ensure that models moving towards production have been reviewed, tested, and approved by the appropriate role.

- **None → Staging:** The data scientist confirms that the model's evaluation metrics exceed predefined thresholds. No unreviewed model should proceed to staging.
- **Staging → Production:** A more senior reviewer (Model Risk Officer, Lead ML Engineer) validates the model's behaviour on a held-out validation set, checks for fairness issues, and confirms the deployment checklist is complete.
- **Production → Archived:** Triggered when a new production model has been confirmed stable. Archiving (not deleting) preserves the model for rollback and audit purposes.

⋆ **Examination Focus:** Key examination topics for Module 4: (a) define the four core MLflow tracking entities and give concrete examples for a given scenario; (b) draw the model lifecycle state diagram with governance checks at each transition; (c) explain which team role is responsible for each transition; (d) compare the limitations of ad-hoc model storage versus a formal model registry (connects back to Module 3). The diagram is highly likely to be tested directly.

Chapter 5

Feature Pipelines and Data Validation

Sessions 9–10 · CLO3, CLO5

Module 5 at a Glance

Sessions: 9 & 10 **Mode:** Hybrid / Lab **CLO:** CLO3, CLO5

Topics: Feature engineering pipelines; training–inference skew; schema validation; pipeline modularisation.

By the end of this module, you should be able to: define training–inference skew and explain its consequences, design a feature pipeline that prevents skew, describe schema validation and why it matters, and explain pipeline modularisation.

5.1 Feature Pipelines in ML Systems

A **feature pipeline** is the set of transformations applied to raw input data to produce the feature vector that is passed to a model. In a production ML system, the feature pipeline is one of the most complex and failure-prone components.

The critical distinction that students must understand is the difference between:

- **Training-time feature pipeline:** Transforms historical data to create the training dataset. Runs once (or periodically for retraining).
- **Inference-time feature pipeline:** Transforms incoming live data to create the feature vector for prediction. Runs for every prediction request.

5.2 Training–Inference Skew

Training–inference skew occurs when the feature transformation logic applied at training time differs from the transformation logic applied at inference time. Even small discrepancies can cause the model to receive inputs at inference time that are systematically different from the inputs it was trained on, leading to degraded and unpredictable predictions.

Training–inference skew is widely considered one of the most insidious failure modes in production ML because it does not cause an obvious error or exception. The model still runs and produces output; the output is simply wrong in a subtle way that may not be immediately apparent.

5.2.1 How Training–Inference Skew Arises

Common causes of training–inference skew:

1. **Separate implementation:** The data scientist writes feature engineering in a Jupyter notebook for training. A software engineer re-implements the "same" logic in Java or another language for production. Small translation errors accumulate.

2. **Normalisation with wrong statistics:** The model was trained on data normalised using the training set mean and standard deviation. At inference time, the same normalisation is applied but using a different (e.g., global or stale) mean and standard deviation.

3. **Feature encoding inconsistency:** Categorical variables are one-hot encoded during training, but the encoding schema at inference time uses a different category ordering or handles unseen categories differently.

4. **Temporal features:** A feature like "days since last purchase" means something different when computed at training time (retrospectively from historical data) versus at inference time (from the live system clock and current transaction records).

5. **Missing value handling:** Training data has missing values imputed using the training set median. At inference time, missing values are imputed with zero or left as null.

5.2.2 Preventing Training–Inference Skew: Shared Pipeline Design

The canonical solution to training–inference skew is the **shared pipeline principle**: the *same code object* that transforms training data must be used, unchanged, to transform inference data. This principle eliminates the possibility of divergence by design.

In `scikit-learn`, this is implemented using `Pipeline` objects:

Concept: Shared Pipeline for Training and Inference

```
TRAINING TIME:
raw_train_data
     |
     v
[Shared Pipeline Object]
|  Step 1: Impute missing values (fit on training data; save statistics)
|  Step 2: Scale numeric features (fit on training data; save mean/std)
|  Step 3: One-hot encode categoricals (fit on training data; save categories)
     |
     v
transformed_train_features  -->  [Train Model]  -->  model.pkl
                                                        +
                                              pipeline.pkl (SAVE THIS TOO)

INFERENCE TIME:
raw_live_data
     |
     v
[Same Pipeline Object loaded from pipeline.pkl]
|  Step 1: Impute (uses saved training statistics -- NOT re-fit)
|  Step 2: Scale (uses saved mean/std -- NOT re-fit)
|  Step 3: One-hot encode (uses saved categories -- NOT re-fit)
     |
     v
transformed_live_features  -->  [Same model.pkl]  -->  prediction
```

The key insight in this diagram is that at inference time, the pipeline object *applies* the statistics learned from training data; it does not *re-learn* statistics from live data. This

guarantees that the feature space seen by the model at inference time is identical to the feature space it was trained on.

5.3 Schema Validation

Schema validation is the process of formally specifying and programmatically enforcing the expected structure, types, and value ranges of a dataset before it is used for training or inference. It catches data quality problems early, before they silently corrupt model behaviour.

A data schema specifies:

- Which columns (features) are expected.
- The data type of each column (integer, float, string, datetime).
- Whether a column is nullable (can contain missing values) and if so, what the maximum acceptable missing rate is.
- The expected value range or set of valid categories for each column.
- Any cross-column constraints (e.g., "end date must be after start date").

5.3.1 Schema Mismatch as a Production Failure Mode

A **schema mismatch** occurs when the data arriving at an ML system does not conform to the schema the model was trained on. Examples:

- A new upstream data source uses the column name `customer_age` where the training data used `age_years`.
- A numeric feature that was always present in training data now sometimes arrives as null due to a change in the data collection system.
- A categorical feature that had 5 valid categories in training now has 6 categories because a new product category was launched.

Without schema validation, these mismatches cause silent failures or runtime exceptions in production. With schema validation, they are caught at the pipeline boundary, triggering an alert before any corrupted prediction reaches a business user.

⋆ **Examination Focus:** Key examination topics for Module 5: (a) define training–inference skew precisely and explain why it is hard to detect; (b) identify specific ways skew could arise in a given scenario (e.g., normalisation statistics mismatch, separate reimplementation); (c) explain the shared pipeline principle and draw the training/inference pipeline diagram; (d) explain schema validation and give examples of schema mismatches. Training–inference skew is a high-priority examination topic.

Chapter 6

Model Packaging and Containerisation

Sessions 11–12 · CLO3

Module 6 at a Glance

Sessions: 11 & 12 **Mode:** Lecture / Lab **CLO:** CLO3

Topics: Model serialisation; packaging models with dependencies; Docker fundamentals for ML; container-based reproducibility.

By the end of this module, you should be able to: explain model serialisation and its pitfalls, describe the role of Docker in ML deployment, draw a Dockerfile structure diagram, and explain how containerisation solves environment inconsistency.

6.1 Model Serialisation

Model serialisation is the process of converting a trained ML model from its in-memory representation (a Python object with learned parameters) into a format that can be written to disk and later reloaded. The serialised file is the *model artefact* that is stored in the model registry and deployed to production.

Common serialisation formats in Python ML:

- **Pickle (.pkl):** Python's native serialisation library. Serialises any Python object. Convenient but carries several risks (see below).
- **Joblib (.joblib):** Similar to Pickle but more efficient for large NumPy arrays; preferred for scikit-learn models.
- **ONNX:** An open, cross-framework model format that allows models trained in Python to be deployed in Java, C++, or other runtimes. Better for cross-language deployment.
- **MLflow Model Format:** MLflow wraps the serialised model with a `MLmodel` metadata file that records the serialisation format, the input signature, and the Python environment required to load the model.

6.1.1 Serialisation Pitfalls

Even when a model trains successfully and produces good metrics, the serialised model artefact can fail to deploy correctly. Common pitfalls:

1. **Library version incompatibility:** A model serialised with scikit-learn version 1.2.0 may fail to load if the production server has scikit-learn version 1.3.0 installed. The internal object structure can change between minor versions.

Mitigation: Record the exact library version at serialisation time (MLflow does this automatically). Ensure the inference environment uses the same version.

2. **Python version incompatibility:** Pickle files are not guaranteed to be compatible across Python versions. A model pickled with Python 3.8 may fail to unpickle with Python 3.10.

   Mitigation: Pin both the Python version and the library versions. Use Docker containers to freeze the entire runtime environment.

3. **Missing custom objects:** If the model uses custom transformer classes (e.g., a bespoke feature engineering class defined in a local module), those class definitions must be present in the loading environment. If the loading environment does not have the same module structure, the unpickling fails.

   Mitigation: Package all custom code alongside the model, or use MLflow's logging to record the code as part of the model artefact. Always deploy model and feature pipeline together.

4. **Platform differences (32-bit vs 64-bit, OS):** Certain numerical types behave differently across operating systems and hardware architectures. Models trained on a 64-bit Linux system may produce subtly different outputs on a 32-bit or Windows system.

   Mitigation: Containerise using Docker with a specific Linux base image.

5. **Forgetting the feature pipeline:** As discussed in Module 5, the model and the feature pipeline must be serialised and deployed together. Deploying the model without the pipeline will cause incorrect predictions due to training–inference skew.

   Mitigation: Wrap the model and pipeline in a single scikit-learn `Pipeline` object before serialisation, so that both are loaded together as a unit.

6.2 Docker for ML: Concepts

Docker is an open-source containerisation platform that allows applications to be packaged with all their dependencies — operating system libraries, language runtime, Python packages, and application code — into a self-contained, portable unit called a **container**.

The key insight for ML practitioners is that a Docker container eliminates the "it works on my machine" problem entirely. A container built on a data scientist's laptop will run identically on a staging server, a production server, or a colleague's workstation, because the container carries its own complete, frozen environment.

6.2.1 Key Docker Concepts

- **Image:** A read-only template from which containers are created. An image is defined by a `Dockerfile` and built using `docker build`. Images are layered, which makes rebuilding efficient (only changed layers are rebuilt).

- **Container:** A running instance of an image. Multiple containers can be spawned from the same image.

- **Dockerfile:** A text file containing the instructions to build a Docker image, layer by layer.

- **Registry:** A storage service for Docker images (Docker Hub is the public registry; organisations often maintain private registries).

6.2.2 Dockerfile Structure for ML Inference Services

A Dockerfile for an ML inference service is built in layers. Each layer adds a component to the environment:

Dockerfile Structure Diagram: ML Inference Service

```
LAYER 1: BASE IMAGE
+----------------------------------------------------+
|  FROM python:3.10-slim                             |
|  (Provides the OS and Python runtime)             |
+----------------------------------------------------+
       |
       v
LAYER 2: SYSTEM DEPENDENCIES
+----------------------------------------------------+
|  RUN apt-get update && apt-get install -y ...      |
|  (OS-level libraries: libgomp for scikit-learn) |
+----------------------------------------------------+
       |
       v
LAYER 3: PYTHON DEPENDENCIES
+----------------------------------------------------+
|  COPY requirements.txt .                          |
|  RUN pip install -r requirements.txt              |
|  (Installs all pinned Python packages)            |
+----------------------------------------------------+
       |
       v
LAYER 4: APPLICATION CODE
+----------------------------------------------------+
|  COPY src/ /app/src/                              |
|  (Copies the feature pipeline and serving code) |
+----------------------------------------------------+
       |
       v
LAYER 5: MODEL ARTIFACT
+----------------------------------------------------+
|  COPY models/pipeline_and_model.pkl /app/model/ |
|  (Copies the serialised model and pipeline)       |
+----------------------------------------------------+
       |
       v
LAYER 6: ENTRYPOINT
+----------------------------------------------------+
|  CMD ["uvicorn", "src.api:app", "--host",         |
|       "0.0.0.0", "--port", "8080"]                |
|  (Defines what runs when the container starts)  |
+----------------------------------------------------+
```

6.2.3 How Containerisation Solves Environment Inconsistency

Without Docker, deploying a model requires the production server to have exactly the right Python version, the right package versions, and the right system libraries. This is fragile: it

breaks whenever the server is updated, whenever a new model is deployed, or whenever the model uses a library not previously on the server.

With Docker:

- The complete environment is *codified in the Dockerfile*, which is version-controlled in Git.
- The image is *built once* and then pushed to a registry.
- At deployment, the server simply *pulls and runs* the image. It does not need any pre-installed Python or ML libraries.
- Different models can use different Python or library versions simultaneously, each isolated in their own container.
- The container is identical in development, staging, and production, eliminating environment-related deployment failures.

⋆ **Examination Focus:** Key examination topics for Module 6: (a) explain model serialisation and identify at least three pitfalls; (b) describe the role of Docker in ML deployment; (c) draw the Dockerfile structure diagram layer by layer and explain what each layer does; (d) articulate clearly how containerisation solves the environment inconsistency problem described in any given scenario. The Dockerfile diagram is a commonly tested visual element.

Chapter 7

Model Serving and Deployment

Sessions 13–14 · CLO3

Module 7 at a Glance

Sessions: 13 & 14 **Mode:** Lecture / Lab **CLO:** CLO3

Topics: Batch vs real-time inference; REST APIs for ML models; deploying models using FastAPI; deployment patterns.

By the end of this module, you should be able to: compare batch and real-time inference across multiple dimensions, describe the architecture of a FastAPI-based inference service, draw a request-response flow diagram, and identify production failure points in the inference pipeline.

7.1 Deployment Patterns: Batch vs Real-Time Inference

Once a model has been packaged (Module 6), it must be *served*: made available to generate predictions. There are two primary serving patterns, each appropriate for different use cases.

7.1.1 Batch Inference

In **batch inference**, the model processes a large set of inputs together, at scheduled intervals (e.g., nightly, weekly). The results are stored in a database or file for downstream consumption. No prediction is made in response to an individual real-time request.

Example use cases: Monthly credit risk scoring for all customers; weekly demand forecasting for product replenishment; daily fraud risk scoring of the previous day's transactions.

7.1.2 Real-Time Inference (Online Inference)

In **real-time inference**, the model generates a prediction *immediately* in response to an individual request, typically within milliseconds. This requires the model to be running as a continuously available service (usually a REST API).

Example use cases: Real-time fraud detection on each credit card transaction; product recommendation on a user's click; patient triage risk scoring at the moment of hospital admission.

7.1.3 Comprehensive Comparison

Batch vs Real-Time Inference: Five-Dimension Comparison

Dimension	Batch Inference	Real-Time Inference
Latency	High: predictions are produced hours or days after data collection; acceptable for non-time-sensitive decisions	Low: sub-second response required; prediction must be ready before the user or system waits
Throughput	Very high: processes millions of records in a single run using parallelised compute	Moderate: optimised for fast individual responses; throughput limited by concurrent capacity
Infrastructure Complexity	Low: scheduled job (e.g., cron); no persistent server needed; compute can scale up for the batch window	High: persistent, highly available server required; load balancers, autoscaling, health checks
Cost	Lower: compute resources used only during the batch window; can use spot/preemptible instances	Higher: server must be running continuously; dedicated resources; redundancy for availability
Use-Case Fit	Periodic decisions where some staleness is acceptable: risk scoring, demand forecasting, reporting	Instantaneous decisions where delay defeats the purpose: fraud detection, recommendation, triage

7.1.4 Choosing Between Batch and Real-Time

The decision is driven by the *business requirement for decision latency*. Ask: "How much time can pass between data collection and decision making before the decision loses value?"

- If the business can tolerate predictions being hours or a day old: **batch inference** is simpler, cheaper, and easier to operate.
- If the decision must be made at the moment of an event (a transaction, a user action, a patient admission): **real-time inference** is required, accepting higher infrastructure cost and complexity.

7.2 REST APIs for ML: The FastAPI Pattern

`FastAPI` is a modern Python web framework for building REST APIs. It is the preferred tool in this course for building ML inference services due to its simplicity, automatic documentation generation, and high performance.

A **REST API** (Representational State Transfer Application Programming Interface) is a web service that clients communicate with using standard HTTP methods. For ML inference, the typical pattern is:

- Client sends an HTTP **POST** request with a JSON body containing the input features.
- The server validates the input, runs the feature pipeline, runs the model, and returns the prediction as a JSON response.

7.2.1 Architecture of a FastAPI ML Inference Service

The inference service sits between the client (an application, dashboard, or other service) and the ML model. Its architecture consists of several components in sequence:

Diagram: FastAPI Request-Response Flow for ML Inference

```
CLIENT (application / dashboard / upstream system)
     |
     |  HTTP POST /predict
     |  Body: {"age": 35, "income": 55000, "loan_amount": 120000}
     v
+--------------------------------------------------+
|  FASTAPI INFERENCE SERVICE                       |
|                                                   |
|  [1] REQUEST PARSING                             |
|      - Parse JSON body from HTTP request         |
|      - Validate data types (e.g., age is int)    |
|                                                   |
|  [2] INPUT VALIDATION  <-- FAILURE POINT 1      |
|      - Pydantic schema: check all required       |
|        fields present, values in range           |
|      - Reject invalid input with HTTP 422        |
|                                                   |
|  [3] FEATURE TRANSFORMATION                      |
|      - Load saved pipeline object                |
|      - Apply imputation, scaling, encoding       |
|        (same pipeline as used in training)       |
|                                                   |
|  [4] MODEL INFERENCE  <-- FAILURE POINT 2       |
|      - Load model from disk (or cache)           |
|      - model.predict(transformed_features)       |
|      - Handle exceptions (e.g., null output)     |
|                                                   |
|  [5] RESPONSE SERIALISATION                      |
|      - Format prediction as JSON                 |
|      - Log: input, output, timestamp, latency    |
|      - Return HTTP 200 with prediction body      |
+--------------------------------------------------+
     |
     |  HTTP 200 OK
     |  Body: {"prediction": "high_risk",
     |         "probability": 0.78}
     v
CLIENT
```

7.2.2 Production Failure Points in the Inference Pipeline

Failure Point 1: Input Validation

This is the most common source of runtime errors in production inference services. Live data is messy: fields may be missing, data types may be wrong, or unexpected values may arrive.

Without explicit input validation, these issues propagate silently into the feature pipeline, causing exceptions or incorrect predictions.

How to handle: Use Pydantic models (integrated with FastAPI) to define the expected input schema. Invalid requests are automatically rejected with a clear HTTP 422 error and an explanation. Log rejected requests for debugging.

Failure Point 2: Model Inference

Model inference can fail due to serialisation issues (covered in Module 6), null or infinity values in features after transformation, or memory exhaustion with large input batches.

How to handle: Wrap the inference step in a try/except block. Return a graceful HTTP 500 error with a structured error message (never expose raw Python tracebacks to clients). Cache the loaded model in memory at startup to avoid slow disk reads on every request.

⋆ **Examination Focus:** Key examination topics for Module 7: (a) compare batch and real-time inference across at least five dimensions in a table; (b) recommend the appropriate deployment pattern for a given scenario with justification; (c) draw the FastAPI request-response flow diagram with all five stages labelled; (d) identify the two primary production failure points (input validation and model inference) and explain how to handle each. The deployment decision table and the flow diagram are both highly likely to appear in the examination.

Chapter 8

CI/CD for Machine Learning

Sessions 15–16 · CLO4

Module 8 at a Glance

Sessions: 15 & 16 **Mode:** Lecture / Lab **CLO:** CLO4

Topics: CI/CD concepts adapted for ML; testing data and models; GitHub Actions for automated training and deployment pipelines.

By the end of this module, you should be able to: explain how CI/CD principles adapt for ML, identify three ways ML CI/CD differs from software CI/CD, design a GitHub Actions pipeline with an evaluation gate, and explain when deployment should be blocked.

8.1 CI/CD: From Software Engineering to Machine Learning

Continuous Integration (CI) is the practice of automatically building and testing code every time a developer pushes a change to a shared repository. This ensures that integration problems are caught immediately rather than accumulating over weeks.

Continuous Delivery (CD) extends CI by automatically packaging and deploying the validated code to a staging or production environment. Together, CI/CD creates a pipeline from code commit to deployed system that is fast, automated, and consistently reliable.

These practices originated in software engineering, where the primary artefact being produced and deployed is *code*. In ML systems, the primary artefact is a *trained model*, which is produced by running code on data. This difference introduces fundamental challenges that require CI/CD to be adapted.

8.2 How ML CI/CD Differs from Software CI/CD

Three Key Differences: Software CI/CD vs ML CI/CD

Dimension	Software CI/CD	ML CI/CD
What changes	Code changes trigger a new deployment; the output is deterministic given the code	Code changes *and* data changes can both trigger retraining; the output (model) depends on both; two identical code commits may produce different models if training data has changed
What is tested	Unit tests, integration tests, and end-to-end tests on code behaviour	Data quality tests (is the training data schema correct?); model quality tests (does the retrained model meet performance thresholds?); behaviour tests (is the model's output distribution reasonable?)
What is deployed	A deterministic binary or service built from code	A model artefact that encodes learned statistical patterns; deploying the "same" code may deploy a materially different model if it was retrained on new data

These differences mean that ML CI/CD requires additional pipeline stages (data validation, model evaluation) and additional governance logic (the *evaluation gate*) that have no direct equivalent in software CI/CD.

8.3 The ML CI/CD Pipeline

A complete CI/CD pipeline for an ML system consists of the following stages, each of which may pass or fail:

Diagram: ML CI/CD Pipeline with GitHub Actions

```
TRIGGER: Code commit to GitHub repository
(or: scheduled data change detected by DVC)
     |
     v
+---------------------------+
|  STAGE 1: Code Quality    |  PASS/FAIL
|  - Linting (flake8)       |
|  - Unit tests (pytest)    |
+----------+----------------+
           |  PASS
           v
+---------------------------+
|  STAGE 2: Data Validation|  PASS/FAIL
|  - Schema check           |
|  - Missing value check    |
|  - Distribution sanity    |
+----------+----------------+
```

```
              |  PASS
              v
   +---------------------------+
   |  STAGE 3: Model Training |
   |  - Run train.py          |
   |  - Log run to MLflow     |
   |  - Register candidate    |
   +----------+----------------+
              |
              v
   +---------------------------+
   |  STAGE 4: EVALUATION GATE|  PASS/FAIL  <-- KEY GOVERNANCE STEP
   |  - Compare new model     |
   |    metrics vs baseline   |
   |  - Check fairness        |
   |  - Check prediction dist |
   +----------+----------------+
              |  PASS (metrics meet thresholds)
              |  FAIL (metrics below threshold) --> STOP, alert team
              v
   +---------------------------+
   |  STAGE 5: Packaging      |
   |  - Build Docker image    |
   |  - Push to registry      |
   +----------+----------------+
              |  PASS
              v
   +---------------------------+
   |  STAGE 6: Deployment     |
   |  - Deploy to staging env |
   |  - Run smoke tests       |
   |  - Promote to production |
   +---------------------------+
```

8.3.1 The Evaluation Gate: The Core Governance Mechanism

The **evaluation gate** is the most important step in an ML CI/CD pipeline. It is an automated comparison of the newly trained model's metrics against a baseline (typically the currently deployed production model). If the new model does not meet the baseline, the pipeline is halted and no deployment occurs.

Conditions that should block deployment:

- **Performance regression:** The new model's primary metric (e.g., F1 score, RMSE) is below the production model's metric by more than a defined tolerance (e.g., more than 2% worse).
- **Fairness violation:** The new model's performance gap between demographic subgroups exceeds an acceptable threshold.
- **Prediction distribution anomaly:** The distribution of the new model's predictions on the validation set is substantially different from expected (e.g., the model now predicts class 1 for 95% of inputs where historically it was 30%).
- **Data validation failure:** The training data itself failed schema validation in Stage 2. A model trained on corrupt data should never be deployed.

- **Latency regression:** The new model is significantly slower at inference time than the production model, potentially violating SLA requirements.

8.3.2 GitHub Actions for ML Pipelines

`GitHub Actions` is the CI/CD automation platform integrated into GitHub. It is configured using YAML workflow files committed to the repository. Each workflow defines a trigger (e.g., push to main branch), a sequence of jobs, and the steps within each job.

Key concepts for ML use:

- **Workflow:** The overall CI/CD specification, defined in `.github/workflows/train_and_deploy.yml`.
- **Trigger:** When the workflow runs. Common triggers: push to a branch, pull request, scheduled time, or manual dispatch.
- **Job:** A set of steps that run on a specified runner environment (e.g., `ubuntu-latest`).
- **Step:** An individual action within a job, such as checking out code, installing dependencies, running a training script, or pushing a Docker image.

⋆ **Examination Focus:** Key examination topics for Module 8: (a) explain three ways ML CI/CD differs from traditional software CI/CD; (b) draw and explain the ML CI/CD pipeline diagram with all six stages; (c) explain the role of the evaluation gate, including at least three failure conditions that should block deployment; (d) describe the components of a GitHub Actions workflow for ML. Use the healthcare triage scenario as a context: explain how the absence of such a pipeline led to an undetected performance degradation.

Chapter 9

Monitoring, Drift, and Retraining

Sessions 17–18 · CLO5

Module 9 at a Glance

Sessions: 17 & 18 **Mode:** Lecture / Lab **CLO:** CLO5

Topics: Model performance monitoring; data drift and concept drift; logging predictions; retraining triggers.

By the end of this module, you should be able to: distinguish data drift from concept drift, explain why both cause silent model degradation, design a monitoring framework with metric categories and alert strategies, and define a retraining trigger policy.

9.1 Why Monitoring Is Non-Negotiable

A deployed ML model is not static. The world it was trained on changes continuously: customer behaviour evolves, product catalogues update, disease patterns shift, economic conditions change. A model that was accurate on its training data will, over time, become less accurate on the data it encounters in production. This degradation is inevitable. What is *not* inevitable is failing to detect it.

Model monitoring is the continuous observation of a deployed model's inputs, outputs, and performance metrics to detect degradation, anomalies, and drift before they cause business harm.

Without monitoring:

- Degradation is detected only when a user or business metric signals a problem — by which point the model has been making poor decisions for weeks or months.
- In regulated environments (banking, healthcare, insurance), this constitutes a governance failure.
- Debugging is far harder because there is no historical log of the model's prediction behaviour.

9.2 Data Drift and Concept Drift

Two types of drift are central to production ML monitoring. They are related but distinct, and require different responses.

9.2.1 Data Drift (Covariate Shift)

> **Data Drift** (also called *covariate shift* or *input drift*) occurs when the statistical distribution of the input features X seen by the model in production changes from the distribution of X in the training data. The relationship between features and the target variable ($P(Y|X)$) remains the same, but the model now encounters input patterns it was not trained on.

Example (Healthcare Triage): An emergency department triage model was trained on patient data from 2022–2024. In early 2025, the hospital opens a new specialist respiratory clinic, leading to a significant increase in patients with respiratory complaints. The distribution of vital signs and symptom inputs shifts substantially from the training distribution. The model has not changed, but the input data has.

Why it causes degradation: The model has implicitly learned patterns that are valid for the historical input distribution. When inputs drift, those patterns may no longer apply. Predictions for newly frequent input patterns may be unreliable.

9.2.2 Concept Drift (Label Drift)

> **Concept Drift** occurs when the relationship between the input features X and the target variable Y changes. That is, $P(Y|X)$ changes over time. Even if the input distribution is stable, the model's predictions become incorrect because the underlying phenomenon it is modelling has changed.

Example (Healthcare Triage): The triage model predicts severity based on symptoms. A new variant of a respiratory illness emerges in 2025 with an atypical presentation — patients with what appear to be mild symptoms based on vital signs are actually at high risk of rapid deterioration. The relationship between the input features (vital signs) and the target (severity outcome) has fundamentally changed. The model, trained on pre-variant data, continues to assign low severity scores to these patients, leading to dangerous under-triage.

Why it is more dangerous: Concept drift is often harder to detect than data drift because the input distribution may look normal. The only way to reliably detect it is to compare model predictions against actual outcomes over time — which requires a ground truth feedback loop (actual outcomes being recorded and matched to predictions).

Data Drift vs Concept Drift: Key Distinction

Property	Data Drift	Concept Drift
What changes	Distribution of input features $P(X)$	Relationship between inputs and target $P(Y\|X)$
Input distribution	Changes (new patterns in X)	May or may not change
Model output	May produce unusual predictions for unfamiliar inputs	Produces systematically wrong predictions even for familiar inputs
Detection method	Statistical tests on input feature distributions (PSI, KL divergence)	Comparison of predictions against actual outcomes (requires ground truth)
Primary risk	Reduced coverage; extrapolation errors	Systematic bias; wrong decisions at scale

9.3 Designing a Monitoring Framework

A complete monitoring framework for a production ML model must address three questions: *What to monitor? How to alert? When to retrain?*

9.3.1 What to Monitor: Three Metric Categories

Category 1: Input Data Quality Metrics

- Missing value rates per feature (is nullability suddenly increasing?)
- Data type violations (are numerics arriving as strings?)
- Schema compliance (are all expected features present?)
- Feature distribution statistics: mean, standard deviation, min/max per feature
- Population Stability Index (PSI): a statistical measure of how much a feature's distribution has shifted relative to the training baseline

Category 2: Prediction Distribution Metrics

- Distribution of predicted classes or scores over time
- Model confidence distribution (are confidence scores clustering unusually at extremes?)
- Prediction volume (is the model receiving an unexpectedly low or high number of requests?)

Category 3: Business-Outcome Proxies (with Ground Truth)

- Actual model accuracy computed on cases where ground truth becomes available (e.g., loan outcomes after 90 days)
- Precision, recall, and F1 on rolling windows of labelled data

- Business KPIs correlated with model quality (e.g., proportion of triage cases escalated; default rate among approved loans)

9.3.2 Logging and Alerting Strategy

Every prediction made by a production model should be logged with at minimum:

- Timestamp
- Input feature values (or a hash if privacy requires)
- Model prediction and confidence score
- Model version identifier
- Inference latency

Logs feed a monitoring dashboard that computes metrics in rolling windows and triggers alerts. Alert thresholds should be set conservatively (catching genuine problems without too many false alarms):

- **Warning alert:** A metric has deviated from its training baseline by more than one defined threshold (e.g., PSI > 0.1 for any feature).
- **Critical alert:** A metric has deviated severely (e.g., PSI > 0.25, or model accuracy has dropped more than 5% from baseline on a labelled sample).

9.3.3 Retraining Trigger Policy

Not every alert warrants immediate retraining. The retraining trigger policy defines the conditions under which retraining should be initiated.

- **Scheduled retraining:** Retrain on a calendar schedule (monthly, quarterly) regardless of detected drift. Useful when ground truth feedback is slow to arrive.
- **Performance-triggered retraining:** Initiate retraining when measured accuracy on a labelled sample drops below a defined threshold. The most precise trigger, but requires ground truth feedback.
- **Drift-triggered retraining:** Initiate retraining when statistical drift measures (PSI, KL divergence) on input features exceed a threshold, even before performance impact is observed. Proactive approach.
- **Human-triggered retraining:** A team member initiates retraining based on domain knowledge of a known external change (new product launch, regulatory change, global event).

Diagram: The Monitoring-Alerting-Retraining Loop

```
PRODUCTION MODEL
(serving predictions continuously)
       |
       | predictions + inputs logged
       v
+-----------------------------+
|   MONITORING DASHBOARD      |
```

```
|   - Input drift metrics     |
|   - Prediction distribution |
|   - Performance metrics     |
+----------+------------------+
           |
     Threshold exceeded?
           |
    YES    |    NO
    +------+        (continue monitoring)
    v
+----------------------------+
|    ALERT SYSTEM            |
|  Warning / Critical alert  |
|  Notifies ML team          |
+----------+------------------+
           |
Evaluate: scheduled / performance / drift / human trigger?
           |
           v
+----------------------------+
|   RETRAINING PIPELINE      |
|   (CI/CD pipeline triggers) |
|   - Collect new data       |
|   - Retrain model          |
|   - Evaluation gate check  |
+----------+------------------+
           |  PASS evaluation gate
           v
NEW MODEL DEPLOYED TO PRODUCTION
(monitoring loop restarts)
```

9.4 The Risk Argument for Monitoring in Safety-Adjacent Systems

For ML systems used in safety-adjacent domains — healthcare, aviation, industrial safety — the argument for comprehensive monitoring and automated retraining pipelines is especially compelling, and not simply a matter of engineering best practice.

Model decay: All deployed models decay over time as the world changes. In a healthcare setting, a triage model's decay may manifest as increasing under-triage of genuinely critical patients, or increasing over-triage (causing resource waste and delayed care for others). This decay is guaranteed to occur; the only question is whether it is detected and corrected in time.

Silent failure: An unmonitored model does not announce when it begins to perform poorly. It continues generating predictions that look exactly like any other prediction. Clinicians trusting the model's recommendations may act on incorrect guidance without any indication that something is wrong. This is the defining danger of unmonitored ML in clinical workflows.

Auditability and regulatory consequences: Healthcare regulators increasingly require that AI-assisted clinical tools maintain logs of all predictions, performance metrics over time, and evidence that models are periodically validated against real patient outcomes. The absence of a monitoring infrastructure means these requirements cannot be met, risking the tool's regulatory approval and the organisation's licence to operate.

⋆ **Examination Focus:** Key examination topics for Module 9: (a) define data drift and concept drift precisely, and explain how each causes model degradation; (b) construct a plausible drift narrative for a given scenario; (c) design a monitoring framework specifying metric categories, logging strategy, and alert levels; (d) define the retraining trigger policy types; (e) construct a risk-based argument for monitoring in a safety context, using the concepts of model decay, silent failure, and auditability. The monitoring loop diagram is likely to appear in the examination.

Chapter 10

End-to-End MLOps Pipeline

Sessions 19–21 · CLO2–CLO5

Module 10 at a Glance

Sessions: 19, 20 & 21 **Mode:** Workshop / Lab **CLO:** CLO2–CLO5

Topics: Design and implementation of a complete MLOps pipeline integrating training, tracking, deployment, and monitoring.

By the end of this module, you should be able to: draw a labelled end-to-end MLOps architecture diagram, explain how each layer connects to the others, and reason about where governance controls should be placed.

10.1 Integrating the Full MLOps Stack

The preceding modules have introduced the components of an MLOps system individually. This module brings them together into a unified, end-to-end architecture that represents the operational reality of a mature ML platform.

An **end-to-end MLOps architecture** is a system in which: data flows continuously from sources through version-controlled pipelines into training; training is automatically triggered and tracked; successful models are automatically packaged and promoted through a governed registry; deployment is automated via CI/CD; and deployed models are continuously monitored with automated retraining feedback loops.

10.2 The Seven Layers of an MLOps Architecture

A complete MLOps architecture can be understood as seven vertically stacked layers, with data flowing upward and governance oversight spanning all layers:

Diagram: End-to-End MLOps Architecture (7 Layers)

```
+==========================================================+
|  LAYER 7: GOVERNANCE & AUDITABILITY OVERLAY              |
|  Access control, audit logs, compliance checks,          |
|  model cards, bias monitoring                            |
|  (Spans ALL layers)                                      |
+==========================================================+
      |                    |                    |
+-----+----+        +------+-----+        +-----+------+
|  BU 1    |        |   BU 2     |        |   BU 3     |
| (Finance)|        | (Ops)      |        | (HR)       |
+-----+----+        +------+-----+        +-----+------+
      |                    |                    |
+==============================================================+
|  LAYER 6: MONITORING & RETRAINING                            |
```

```
|  Prediction logging, drift detection, performance metrics,  |
|  alerting, scheduled/triggered retraining                   |
+=============================================================+
      |
+=============================================================+
|  LAYER 5: DEPLOYMENT & SERVING                              |
|  Docker containers, FastAPI inference services,             |
|  batch jobs, load balancers, staging/production envs        |
|                      ^                                      |
|             [GOVERNANCE CHECK 1]                            |
|        (Evaluation gate: metrics, fairness, latency)        |
+=============================================================+
      |
+=============================================================+
|  LAYER 4: CI/CD AUTOMATION                                  |
|  GitHub Actions workflows: code quality, data validation,   |
|  training trigger, model packaging, auto-deployment         |
+=============================================================+
      |
+=============================================================+
|  LAYER 3: MODEL REGISTRY                                    |
|  MLflow registry: versioned models, lifecycle stages,       |
|  (None -> Staging -> Production -> Archived)                |
|                      ^                                      |
|             [GOVERNANCE CHECK 2]                            |
|        (Stage transition approval: MLOps lead/risk officer) |
+=============================================================+
      |
+=============================================================+
|  LAYER 2: EXPERIMENT TRACKING                               |
|  MLflow experiments: runs, parameters, metrics, artifacts   |
|  Git commit hash + DVC pointer per run (full lineage)       |
+=============================================================+
      |
+=============================================================+
|  LAYER 1: DATA & CODE VERSIONING                            |
|  Git (code), DVC (data & models), config files              |
|  Multiple data sources: databases, APIs, files              |
+=============================================================+
```

10.3 How the Layers Connect

1. **Layer 1 feeds Layer 2:** When a data scientist runs a training script, the Git commit hash and DVC dataset pointer are automatically logged with the experiment run, creating a permanent linkage between code, data, and experiment.

2. **Layer 2 feeds Layer 3:** The best-performing run from Layer 2 is registered in the Model Registry (Layer 3) as a new candidate model version. The experiment run ID is embedded in the registry record, maintaining full lineage.

3. **Layer 3 feeds Layer 4:** Stage transitions in the registry (e.g., candidate → staging) can trigger CI/CD workflows in Layer 4, automatically packaging and deploying the model to a staging environment.

4. **Layer 4 enforces governance at Governance Check 1:** Before deployment, the CI/CD pipeline runs the evaluation gate, blocking any model that does not meet quality

and fairness thresholds.

5. **Layer 4 feeds Layer 5:** A passed evaluation gate triggers Docker image build and deployment to the serving infrastructure.
6. **Layer 5 feeds Layer 6:** All predictions made by the serving layer are logged to the monitoring system.
7. **Layer 6 feeds back to Layer 1:** When monitoring detects drift or performance degradation, the retraining trigger policy fires, initiating a new training run at Layer 1. The loop closes.

10.4 Governance Integration Points

In the architecture above, governance is not a separate activity at the end of the pipeline; it is integrated at two explicit checkpoints and as a continuous overlay:

- **Governance Check 1 (Evaluation Gate, Layer 4):** Automated quality, fairness, and performance gating before every deployment. No human approval needed for routine retraining if thresholds are met; human review required if metrics are borderline or fairness flags arise.
- **Governance Check 2 (Registry Stage Transition, Layer 3):** Human approval by a designated Model Risk Officer or senior ML Engineer before a model moves from Staging to Production. This is the primary human-in-the-loop governance moment.
- **Governance Overlay (Layer 7):** Continuous audit logging of all pipeline events, model predictions, and stage transitions; model cards documenting each production model's intended use, training data, performance characteristics, and known limitations.

⋆ **Examination Focus:** Key examination topics for Module 10: (a) draw the seven-layer end-to-end MLOps architecture diagram, labelling each layer and marking the two governance check points; (b) explain how adjacent layers connect and what triggers data flow between them; (c) explain the role of the governance overlay and what it spans. This is the integrating module of the course and the architecture diagram is the most important single diagram to master for the Exams.

Chapter 11

Cloud MLOps Concepts

Session 22 · CLO1, CLO3

Module 11 at a Glance

Sessions: 22 **Mode:** Lecture **CLO:** CLO1, CLO3

Topics: Conceptual overview of cloud-native MLOps architectures; scaling considerations; cost and deployment trade-offs.

By the end of this module, you should be able to: map the open-source MLOps stack to equivalent managed cloud services, explain the trade-offs between self-managed and cloud-managed MLOps infrastructure, and identify key scaling considerations.

11.1 Cloud MLOps: The Conceptual Shift

All previous modules have used open-source tools (Git, DVC, MLflow, Docker, FastAPI, GitHub Actions) to build MLOps capabilities. Cloud platforms offer *managed services* that provide equivalent functionality without the operational burden of self-managing the underlying infrastructure.

The key trade-off is **operational control vs operational overhead**: self-managed open-source stacks give full control over configuration and data residency but require the team to maintain the infrastructure; managed cloud services reduce operational burden but introduce vendor lock-in, cost at scale, and potential data sovereignty concerns.

11.2 Cloud-to-Open-Source Mapping

MLOps Capability: Open-Source vs Cloud Equivalents

MLOps Capability	Open-Source (This Course)	AWS	GCP	Azure
Experiment Tracking	MLflow	SageMaker Experiments	Vertex AI Experiments	Azure ML
Model Registry	MLflow Registry	SageMaker Model Registry	Vertex AI Model Registry	Azure ML Registry
Data Versioning	DVC	S3 + versioning	GCS + versioning	Azure Blob Storage
CI/CD	GitHub Actions	CodePipeline	Cloud Build	Azure DevOps
Model Serving	FastAPI + Docker	SageMaker Endpoints	Vertex AI Endpoints	Azure ML Endpoints
Monitoring	Custom (Evidently)	SageMaker Model Monitor	Vertex AI Model Monitoring	Azure ML Monitor
Orchestration	Manual / Scripts	Step Functions	Vertex AI Pipelines	Azure ML Pipelines

11.3 Scaling Considerations

When ML systems scale from a handful of models to hundreds, new operational challenges emerge:

- **Infrastructure as Code:** At scale, MLOps infrastructure itself must be version-controlled and automatically provisioned (Terraform, Pulumi).
- **Compute elasticity:** Training large models requires GPU clusters that scale up for a training job and scale down afterwards. Cloud platforms provide this elasticity; self-managed clusters do not.
- **Cost management:** Cloud compute costs for ML can escalate rapidly. Cost controls (spend limits, spot instance policies, experiment budgeting) become essential.
- **Latency at scale:** Serving thousands of concurrent real-time inference requests requires load balancers, autoscaling groups, and potentially edge deployment for the lowest latency.
- **Data governance at scale:** When data crosses regions or cloud providers, regulatory requirements (GDPR, DPDP) must be actively managed.

Chapter 12

Governance, Ethics, and Industry Readiness

Sessions 23–24 · CLO6

Module 12 at a Glance

Sessions: 23 & 24 **Mode:** Lecture **CLO:** CLO6

Topics: Responsible AI; auditability; compliance; MLOps roles and career pathways; end-to-end pipeline review.

By the end of this module, you should be able to: apply the four dimensions of Responsible AI, distinguish auditability from explainability, design an audit trail, describe MLOps organisational roles in a RACI framework, and analyse proxy discrimination and governance failures.

12.1 Responsible AI in Production ML

The deployment of ML models at scale creates ethical obligations that go beyond technical performance. **Responsible AI** is the practice of designing, deploying, and monitoring AI systems in ways that are fair, accountable, transparent, and privacy-respecting — sometimes abbreviated as the **FAPT framework**.

Each dimension of FAPT has distinct implications for MLOps practice:

12.1.1 Fairness

> **Fairness** in ML means that a model's predictions and outcomes should not systematically disadvantage individuals or groups based on protected characteristics such as race, gender, age, religion, disability, or (often) geographic origin.

Fairness is a multi-dimensional concept with several formal definitions that can conflict with each other:

- **Demographic parity:** The model's positive prediction rate is equal across protected groups.
- **Equalised odds:** The model's true positive rate and false positive rate are equal across protected groups.
- **Individual fairness:** Similar individuals should receive similar predictions.

Fairness violations in production ML typically arise from three sources:

1. **Historical bias in training data:** If historical human decisions embedded in the training data were discriminatory, the model learns to replicate that discrimination.

2. **Proxy features:** A model may not directly use a protected attribute (e.g., race) as a feature, but may use a *proxy feature* (e.g., postal code) that is highly correlated with race, indirectly encoding the discrimination. This is the phenomenon of **proxy discrimination**.

3. **Feedback loops:** Model predictions that influence future data collection (e.g., a policing model that targets certain areas more heavily, generating more arrest data from those areas, which reinforces the model's targeting) create compounding fairness problems.

12.1.1.1 Proxy Discrimination: A Detailed Analysis

Proxy discrimination is particularly dangerous because it can occur even when the model developer has explicitly excluded protected attributes from the feature set. The developer may believe they have built a neutral model, but the model has learned an equivalent discrimination through correlated variables.

Example (Supplier Risk Scoring): A model designed to score supplier reliability uses features including postal code, historical payment delays, and contract size. No ethnicity or demographic variable is explicitly included. However, suppliers from certain postal codes are predominantly owned by individuals from particular ethnic minority communities. The model assigns higher risk scores to suppliers from these codes — not because of ethnicity per se, but because postal code acts as a proxy. The discriminatory effect is identical to explicit ethnic discrimination.

Four-Step Remediation for Proxy Discrimination:

1. **Data Audit:** Conduct a systematic analysis of correlations between all features in the model and protected demographic attributes. Identify which features are acting as proxies. This requires obtaining (often through external datasets or surveys) demographic information about the affected population.

2. **Model Re-Evaluation:** Retrain the model excluding identified proxy features, or applying debiasing techniques (reweighting, adversarial debiasing). Evaluate the model's fairness metrics (demographic parity, equalised odds) across demographic subgroups on a labelled validation dataset. Document all findings.

3. **Deployment Decision:** Based on re-evaluation results, make one of three decisions: (a) deploy the debiased model if fairness metrics meet thresholds; (b) delay deployment pending further remediation if the problem is partially resolved; (c) suspend deployment if the discrimination cannot be adequately mitigated and the business or regulatory risk is unacceptable.

4. **Ongoing Monitoring:** Add fairness metrics to the production monitoring dashboard (Module 9). Disaggregate performance metrics by demographic group. Alert when performance gaps between groups exceed defined thresholds. Schedule periodic fairness audits as part of the model's retraining cycle.

12.1.2 Privacy

> **Privacy** in ML means that models should be trained and deployed in ways that protect the personal data of the individuals whose data was used in training and who are affected by model predictions.

Key privacy risks in ML:

- **Membership inference attacks:** An adversary may be able to determine whether a specific individual's data was used in training a model by querying the model strategically.
- **Excessive data retention:** Training data containing personal information may be retained longer than necessary, violating data minimisation principles under GDPR or DPDP.
- **Re-identification:** "Anonymised" training data may be re-identified when combined with other datasets.

MLOps controls for privacy:

- Data access controls and role-based permissions on training data
- Data retention policies enforced at the pipeline level
- Differential privacy techniques for sensitive training sets
- Privacy Impact Assessments before deploying models on personal data

12.1.3 Accountability

> **Accountability** means that there are clearly identified individuals or teams who are responsible for the behaviour of a deployed ML model, and that mechanisms exist to hold them responsible when the model causes harm.

Accountability requires:

- Clear assignment of the *model owner*: the individual responsible for a model's performance, fairness, and compliance throughout its lifecycle.
- A **model card**: a standardised documentation artefact that records a model's intended use, training data characteristics, performance metrics, fairness evaluations, and known limitations. Model cards provide the basis for informed accountability.
- An audit trail (see Section 12.3) that records every decision in the model's lifecycle, allowing accountability to be assigned after the fact.

12.1.4 Transparency

> **Transparency** means that the logic of ML model decisions should be understandable to the degree necessary for their context, and that the existence and use of ML models in decision-making should be disclosed to affected parties.

Transparency in MLOps has two dimensions:

- **Internal transparency:** The data science team and business stakeholders can understand why the model makes specific predictions. This is the domain of *model explainability* (SHAP values, feature importance, LIME).

- **External transparency:** Customers, patients, or other affected parties are informed when an ML model is making decisions that affect them, and are told how to seek human review if they dispute the decision.

12.2 Model Auditability vs Model Explainability

A critical conceptual distinction that students frequently confuse:

Auditability vs Explainability

Concept	Definition	Example
Model Explainability	The ability to understand *why a specific prediction was made* — to identify which input features most influenced a particular output	"This loan application was rejected because the applicant's debt-to-income ratio (feature weight: 0.42) and employment tenure (weight: 0.31) were below thresholds."
Model Auditability	The ability to verify *how the model was built and deployed* — to reconstruct the complete provenance of a model: what data was used, what code was run, who approved it, and when	"Model v3.2 was trained on dataset snapshot 2025-Q4 (DVC hash: a3f9), using code at commit d72bc3, approved by risk officer on 2025-11-14, and deployed on 2025-11-16."

Explainability answers the question: *"Why did this model make this specific prediction?"*
Auditability answers the question: *"How was this model created, validated, and deployed?"*

Both are necessary. Explainability is a technical property of the model itself; auditability is an organisational property of the MLOps process.

12.3 Designing a Minimum Audit Trail

Every ML model deployed in a production system should have a minimum audit trail — a set of artefacts and logs that, taken together, allow a complete reconstruction of the model's provenance and operational history.

Minimum Audit Trail: Five Essential Artefacts

Artefact	What It Records	Audit Question It Answers
1. Git Commit Log	Exact version of all code used to train the model, with authorship and timestamp	"What code built this model and who wrote it?"
2. DVC Dataset Record	Content hash and metadata of the exact training dataset version	"What data was the model trained on?"
3. MLflow Experiment Run	All parameters, metrics, and artifacts from the training run	"What were the model's hyperparameters and evaluation metrics?"
4. Model Registry History	Log of all stage transitions, including who approved each transition and when	"Who approved this model for production and when?"
5. Production Prediction Logs	Every prediction made in production: input, output, model version, timestamp	"What decisions has this model made, and are they consistent with its validated behaviour?"

12.4 MLOps Organisational Roles

Mature ML organisations define specific roles to ensure that the various responsibilities of the MLOps lifecycle are clearly owned. The absence of clear role definitions is a major source of governance failure.

Core MLOps Roles: Responsibilities and Interactions

Role	Primary Responsibilities
Data Scientist	Problem framing, feature engineering, model design, experiment tracking, model evaluation, initial documentation
ML Engineer	Pipeline development, CI/CD configuration, model packaging, deployment automation, integration testing
Data Engineer	Data ingestion pipelines, data quality, schema management, data versioning, feature store management
MLOps Platform Engineer	Infrastructure management (cluster, registry, monitoring tools), developer tooling, platform reliability
Model Risk Officer	Governance oversight, stage transition approvals, fairness audits, regulatory compliance, model card review

12.4.1 RACI Framework for ML Model Deployment

A **RACI matrix** assigns each activity to one of four relationship types: **R**esponsible (does the work), **A**ccountable (owns the outcome), **C**onsulted (provides input), **I**nformed (kept in the loop).

RACI Matrix: Model from Experimentation to Production

Activity	Data Sci.	ML Eng.	Data Eng.	Platform Eng.	Risk Officer
Feature engineering	R/A	C	C	I	I
Experiment tracking	R/A	C	I	I	I
Model evaluation	R/A	C	I	I	C
Pipeline development	C	R/A	C	C	I
Registry promotion (Staging)	A	R	I	I	C
Registry promotion (Prod.)	C	C	I	I	R/A
CI/CD configuration	C	R/A	I	C	I
Monitoring setup	C	R	C	A	C
Fairness audit	C	C	I	I	R/A

⋆ **Examination Focus:** Key examination topics for Module 12: (a) explain the four FAPT dimensions of Responsible AI with concrete examples in a specific context (employee attrition, supplier risk, clinical decision support); (b) distinguish model auditability from model explainability with clear definitions and examples; (c) list the five minimum audit trail artefacts and explain what governance question each answers; (d) describe MLOps roles and draw or populate the RACI matrix; (e) analyse proxy discrimination in a given scenario and propose a four-step remediation plan. This is the governance and ethics module and all topics here are high-weight examination material.

Appendix: Key Concepts Glossary

Artifacts (MLflow): Files produced by or associated with a training run, such as model files, plots, or data schemas.

Auditability (Model): The ability to reconstruct the complete provenance and history of a deployed model — what data, code, and approvals produced it.

Batch Inference: A serving pattern in which a model processes large datasets at scheduled intervals rather than in real time.

Branch (Git): A separate line of development that diverges from the main codebase, enabling parallel experimentation without interference.

CI/CD: Continuous Integration and Continuous Delivery. Automated pipelines that build, test, and deploy code (and, in MLOps, models) on every change.

Concept Drift: A change in the relationship between input features and the target variable $P(Y|X)$, causing a model's predictions to become systematically incorrect even if input distributions are stable.

Container (Docker): A running instance of a Docker image that encapsulates a complete, isolated application environment.

Data Drift: A change in the statistical distribution of input features $P(X)$ in production compared to the training distribution.

Data Versioning: The practice of creating immutable, hash-identified snapshots of datasets to enable reproducibility and auditability.

Dockerfile: A text file specifying layered instructions to build a Docker image.

Docker Image: A read-only, layered template from which Docker containers are instantiated.

DVC (Data Version Control): An open-source tool that versions large data files via content hashes and pointer files, working alongside Git.

Evaluation Gate: An automated step in a CI/CD pipeline that blocks deployment if the new model's metrics do not meet defined thresholds.

Experiment (MLflow): A named collection of related training runs in the MLflow tracking server.

Experiment Tracking: The practice of automatically recording every training run with its parameters, metrics, and artifacts.

Explainability (Model): The ability to understand why a specific prediction was made — which features most influenced the output.

FAPT Framework: Fairness, Accountability, Privacy, Transparency. The four dimensions of Responsible AI.

Fairness (ML): The property that a model does not systematically disadvantage groups based on protected characteristics.

FastAPI: A Python web framework for building high-performance REST APIs, used in this course for ML inference services.

Feature Pipeline: The sequence of transformations applied to raw data to produce model-ready feature vectors.

GitHub Actions: A CI/CD automation platform integrated with GitHub, configured via YAML workflow files.

Git: A distributed version control system for tracking code changes.

Governance (ML): The policies, processes, roles, and controls that ensure ML models are built, deployed, and operated responsibly.

Hyperparameter: A configuration value set before training that controls the learning algorithm (e.g., learning rate, tree depth). Logged as a *parameter* in MLflow.

Inference: The process of running a trained model on new input data to generate predictions.

Maturity Level (MLOps): A classification of an organisation's MLOps capabilities from Level 0 (manual/notebook) to Level 2 (fully automated CI/CD).

Metrics (MLflow): Numeric values recorded during or after a training run to measure model performance (e.g., accuracy, RMSE).

MLflow: An open-source platform for managing ML experiments, model versions, and deployment, used throughout this course.

ML Lifecycle: The complete set of stages from

problem framing through data collection, training, evaluation, deployment, and monitoring.

ML Model: A mathematical function that maps inputs to outputs, parameterised by learned weights.

ML System: The complete sociotechnical infrastructure surrounding an ML model, including data pipelines, serving infrastructure, monitoring, and governance.

MLOps: Machine Learning Operations. The discipline of applying software engineering best practices to the complete ML lifecycle.

Model Card: A standardised document describing a model's intended use, training data, evaluation results, fairness assessment, and known limitations.

Model Decay: The gradual degradation of a deployed model's performance as the world changes and the training data becomes less representative.

Model Registry: A centralised catalogue of trained ML models with versioning and lifecycle management.

Parameters (MLflow): Input configuration values (hyperparameters) logged for a training run.

Pickle (.pkl): Python's native serialisation format, widely used for saving ML models.

Population Stability Index (PSI): A statistical measure used to quantify how much the distribution of a variable has shifted between a training baseline and a production sample.

Proxy Discrimination: A form of algorithmic bias in which a model discriminates against protected groups not through direct use of a protected attribute, but through a correlated proxy feature.

RACI Matrix: A responsibility assignment framework: Responsible, Accountable, Consulted, Informed.

Real-Time Inference: A serving pattern in which the model generates predictions immediately in response to individual requests.

Reproducibility: The property that the same code, data, environment, and configuration always produce the same results.

REST API: Representational State Transfer Application Programming Interface. A web service architecture using standard HTTP methods.

Run (MLflow): A single execution of a training script, uniquely identified and containing logged parameters, metrics, and artifacts.

Schema Validation: The programmatic enforcement of expected data structure, types, and value ranges at pipeline boundaries.

Serialisation (Model): Converting a trained model from an in-memory Python object to a format storable on disk.

Silent Failure: A mode of ML system failure in which the model continues to produce predictions, but the predictions are incorrect, without generating any obvious error signal.

Training–Inference Skew: The condition in which the feature transformation logic at training time differs from the transformation logic at inference time, leading to degraded predictions.

Virtual Environment: An isolated Python installation with specific package versions, used to prevent environment variability between team members and systems.